

SIMATS ENGINEERING

ASSESSMENT TOOL 1

**COURSE CODESA0713 COURSE NAME: COMPUTER
NETWORKS**

COURSE OUTCOME COVERED

**CO2: Configure IP addressing schemes, subnetting, and basic routing techniques
using simulation tool, for efficient network design and topology. (BL3)**

Assessment Tool Weightage: 40%

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

**I D. Anand Paul(192425297) (Name / Reg No) certify that this submission is my original
work and that I have adhered to the guidelines specified for this assessment**

**I understand that any violation of academic integrity rules will result in disciplinary
action.**

Signature of the student: Anand Paul

Q1. CRC Error Detection

Given:

Message = 11100011

Generator Polynomial $G(x) = X^3 + 1 \rightarrow$ Divisor = 1001 (degree $r = 3$)

Step 1: Append r zeros to message

Since divisor has 4 bits (degree 3), append 3 zeros to message.

Augmented message = 11100011 000 = 11100011000

Step 2: Perform Modulo-2 (XOR) Division

11100011000
÷ 1001
1110
1001
0111 0 → bring down bit → 1110
1001
0111 0 → bring down bit → 1110
1001
0111 1 → bring down bit → 1111
1001
0110 1 → bring down bit → 1101
1001
0100 0 → bring down bit → 1000
1001
0001 0 → bring down bit → 0010
0010 0 → bring down bit → 0100

Remainder (CRC) = **100**

Step 3: Form Transmitted Codeword

Transmitted message = Original message + CRC
 = 11100011 + 100
 = **11100011100**

(Verified: dividing 11100011100 by 1001 gives remainder 000 → confirms divisibility)

Final Answer:

Transmitted Message = 11100011100

Q1 (ii) Error Detection at Receiver

Leftmost bit of transmitted codeword (11100011100) is inverted:

Received message = **0**1100011100

Receiver divides received message by 1001:

01100011100 ÷ 1001

Performing the same modulo-2 division:

0110
0000(no xor,leading 0)
0110 0 → 1100
1001
0101 0 → 1010
1001
0011 1 → 0111
0111(no xor)
0111 1 → 1111
1001
0110 1 → 1101

1001
0100 0 → 1000
1001
0001 0 → 0010

Final Remainder = **010** (Non-zero)

Explanation:

Since the received codeword was originally exactly divisible by the generator polynomial (remainder = 0), any single-bit error introduces a non-zero remainder at the receiver. Here the receiver's remainder = 010 $\neq$ 0, hence the receiver knows the frame has been corrupted during transmission and requests retransmission.

Final Answer:

Remainder = 010 ($\neq$ 0) → Error Detected

Q2 (a). Throughput Calculation

Given:

Bandwidth = 10 Mbps

Frames/min = 12,000

Bits/frame = 10,000

Formula:

Throughput = (Frames per second $\times$ Bits per frame)

Substitution:

Frames per second = 12000 / 60 = 200 frames/sec

Throughput = 200 $\times$ 10,000 = 2,000,000 bps

Calculation:

Throughput = 2,000,000 bps = 2 Mbps

Final Answer:

Throughput = 2 Mbps

Q2 (b). Channel Capacity (Shannon's Theorem)

Given:

Bandwidth $B = 3000$ Hz

SNR = 3162

Formula:

$$C = B \log_2(1 + \text{SNR})$$

Substitution:

$$C = 3000 \times \log_2(1 + 3162)$$

$$C = 3000 \times \log_2(3163)$$

Calculation:

$$\log_2(3163) \approx 11.62$$

$$C = 3000 \times 11.62$$

$$C \approx 34,860 \text{ bps}$$

Final Answer:

$$C \approx 34.86 \text{ kbps (34,860 bps)}$$

Q3. Hamming (12,8) Error Correcting Code

Concept:

Hamming (12,8) code takes 8 data bits ($m=8$) and adds 4 redundant/parity bits ($r=4$), such that $2^r \geq m+r+1 \rightarrow 2^4=16 \geq 13$ (satisfied). Total codeword length = 12 bits. Parity bits are placed at positions that are powers of 2 (1, 2, 4, 8); remaining positions hold data bits.

Bit Position Table:

Position	1	2	3	4	5	6	7	8	9	10	11	12
Bit Type	P1	P2	D1	P4	D2	D3	D4	P8	D5	D6	D7	D8

Given Data (assume 8-bit patient data) = 1 0 1 1 0 0 1 0

Assigning: D1=1, D2=0, D3=1, D4=1, D5=0, D6=0, D7=1, D8=0

Step 1: Calculate Parity Bits

P1 (covers 1,3,5,7,9,11): $D1+D2+D4+D7 = 1+0+1+1 = 1$

P2 (covers 2,3,6,7,10,11): $D1+D3+D4+D7 = 1+1+1+1 = 0$

P4 (covers 4,5,6,7,12): $D2+D3+D4+D8 = 0+1+1+0 = 0$

P8 (covers 8,9,10,11,12): $D5+D6+D7+D8 = 0+0+1+0 = 1$

Step 2: Final Transmitted Codeword

Pos	1	2	3	4	5	6	7	8	9	10	11	12
Bit	1	0	1	0	0	1	1	1	0	0	1	0

Codeword = 1 0 1 0 0 1 1 1 0 0 1 0

Step 3: Simulate Transmission Error (interference flips bit 7)

Received Codeword = 1 0 1 0 0 1 0 1 0 0 1 0

Step 4: Receiver Recomputes Parity Checks (Syndrome)

C1 = bits(1,3,5,7,9,11) = $1+1+0+0+1 = 1$

C2 = bits(2,3,6,7,10,11) = $0+1+1+0+1 = 1$

C4 = bits(4,5,6,7,12) = $0+0+1+0+0 = 1$

C8 = bits(8,9,10,11,12) = $1+0+0+1+0 = 0$

Syndrome = C8 C4 C2 C1 = 0 1 1 1 = **7 (decimal)**

Step 5: Correction

Since syndrome $\neq 0$, error exists at position 7. Receiver flips bit 7 back:

0 $\rightarrow$ 1, restoring correct codeword automatically — no retransmission needed, ensuring doctors get accurate real-time readings.

Final Answer:

Error detected & corrected at bit position 7

Corrected Codeword = 101001111010... (i.e., original transmitted codeword restored)

Q4 (a). Subnetting

Given:

Base network: 172.16.10.0/24

New subnet mask: /27

Step 1: Number of Subnets

Bits borrowed = $27 - 24 = 3$

Number of subnets = $2^3 = 8$ subnets

Step 2: Hosts per Subnet

Host bits = $32 - 27 = 5$

Usable hosts/subnet = $2^5 - 2 = 30$ hosts

Step 3: Subnet Block Size

Block size = $256 - 224(\text{mask}) = 32$

(Subnet mask /27 = 255.255.255.224)

Subnet Table:

Subnet No.	Network Address	Usable Range	Broadcast Address
1	172.16.10.0	.1 – .30	172.16.10.31
2	172.16.10.32	.33 – .62	172.16.10.63
3	172.16.10.64	.65 – .94	172.16.10.95
4	172.16.10.96	.97 – .126	172.16.10.127
5	172.16.10.128	.129 – .158	172.16.10.159
6	172.16.10.160	.161 – .190	172.16.10.191

Subnet No.	Network Address	Usable Range	Broadcast Address
7	172.16.10.192	.193 – .222	172.16.10.223
8	172.16.10.224	.225 – .254	172.16.10.255

Step 4: Locate 172.16.10.78

78 lies between 64 – 95 → belongs to **Subnet 3**

Network Address = 172.16.10.64

Broadcast Address = 172.16.10.95

Step 5: Check 172.16.10.95

172.16.10.95 = Broadcast address of the SAME subnet (172.16.10.64/27), so **yes, it lies within the same subnet range.**

Subnets = 8, Usable Hosts/Subnet = 30

172.16.10.78 → Subnet 172.16.10.64/27

Network Addr = 172.16.10.64, Broadcast = 172.16.10.95

172.16.10.95 → within same subnet (it is the broadcast address)

Q4 (b). IPv6 Addressing

Given:

Address = 2001:0db8:0000:0000:0000:ff00:0042:8329

Step 1: Compression

Remove leading zeros in each group:

2001: db8 : 0 : 0 : 0 : ff00: 42 : 8329

Replace consecutive all-zero groups with "::":

Compressed = 2001:db8::ff00:42:8329

Step 2: Expansion (back to full form)

Restore each group to 4 hex digits and replace ":" with correct number of zero groups:

Expanded = 2001:0db8:0000:0000:0000:ff00:0042:8329

Step 3: Network Prefix for /64

First 64 bits (4 leftmost groups) form the network prefix:

Network Prefix = **2001:0db8:0000:0000: :/64** (or 2001:db8::/64)

Step 4: Number of Host Addresses

Host bits = $128 - 64 = 64$

Total Host Addresses = 2^{64}

Final Answer:

Compressed = 2001:db8:ff00:42:8329

Expanded = 2001:0db8:0000:0000:0000:ff00:0042:8329

Prefix (/64) = 2001:0db8:0000:0000: :/64

Total Hosts = 2^{64} addresses

Q5. Flow Control Problem – University Exam Server Scenario

Identification of Primary Networking Problem:

The scenario describes a classic **Flow Control problem**, arising due to a **speed mismatch between a fast sender and a slow receiver**. The examination server (10 Gbps) transmits data much faster than the receiving student devices (100 Mbps, 20 Mbps, 5–10 Mbps) can accept and process it.

Explanation of the Mismatch:

1. Sender Rate vs Receiver Rate:

The server pushes question paper data continuously at high speed without checking whether the receiver's buffer has space. Since receiving devices operate on much lower bandwidth links, their input buffers fill up faster than the application layer can process the incoming data.

2. **Buffer Overflow:**

Once the receiver's buffer becomes full, any additional incoming packets are dropped, since there is no space to store them temporarily before processing.

3. **Retransmissions:**

Dropped packets are detected by the receiver (or through timeout at sender side), triggering retransmission requests. This increases unnecessary traffic on an already congested network.

1. **Delayed Acknowledgements:**

As the receiver device is overwhelmed with incoming data, it cannot generate ACKs promptly, causing the sender (if using a fixed/large window) to continue sending more data — worsening the overflow condition.

2. **Portal Freezing:**

With continuous packet loss and retransmission cycles, the effective throughput drops drastically, causing the student's exam portal to appear frozen or unresponsive.

How this differs from Congestion Control:

Flow control deals with sender–receiver speed mismatch (end-to-end), whereas congestion control deals with overload within the network (routers/links). In this scenario, both issues coexist because 8,000 simultaneous connections also strain intermediate network resources, but the description specifically emphasizes **receiver capacity mismatch**, making Flow Control the primary issue.

Suggested Solution:

- Implement **Sliding Window Flow Control** (as used in TCP) so that the sender only transmits as much data as advertised by the receiver's window size (rwnd).
- Use **TCP's dynamic window adjustment**, where the receiver advertises available buffer space in every ACK, forcing the sender to slow down when buffers are near full.
- Employ **buffering and pacing mechanisms** on the server side to avoid overwhelming low-bandwidth clients.
- Introduce **congestion control algorithms** (e.g., slow start, congestion avoidance) to reduce network-wide overload from thousands of parallel connections.

